

Exercice 1 - Forward Pass et analyse des parametres

Question 1.1 - Calcul du Forward Pass

Donnees du probleme :

Entrees : $x_1 = 0.5$, $x_2 = 1$, $x_3 = 2$

Poids couche 1 : $W_1 = [[0.2 ; 0.5], [0.3 ; 0.1], [0.4 ; 0.2]]$, $b_1 = [0.1 ; 0.2]$

Poids couche 2 : $W_2 = [[0.7], [0.3]]$, $b_2 = [0.1]$

Etape 1 : Calcul de $z_L = x * W_L + b_L$

Pour le premier neurone de la couche cachee :

$$z_{L11} = (0.5 * 0.2) + (1 * 0.3) + (2 * 0.4) + 0.1 = 0.1 + 0.3 + 0.8 + 0.1 = 1.3$$

Pour le deuxieme neurone de la couche cachee :

$$z_{L12} = (0.5 * 0.5) + (1 * 0.1) + (2 * 0.2) + 0.2 = 0.25 + 0.1 + 0.4 + 0.2 = 0.95$$

Resultat : $z_L = [1.3 ; 0.95]$

Etape 2 : Application de $a_L = \text{ReLU}(z_L)$

La fonction ReLU est definie par $\text{ReLU}(x) = \max(0, x)$.

$$a_{L11} = \text{ReLU}(1.3) = 1.3$$

$$a_{L12} = \text{ReLU}(0.95) = 0.95$$

Resultat : $a_L = [1.3 ; 0.95]$

Etape 3 : Calcul de $z_{\text{out}} = a_L * W_{\text{out}} + b_{\text{out}}$

$$z_{\text{out}} = (1.3 * 0.7) + (0.95 * 0.3) + 0.1 = 0.91 + 0.285 + 0.1 = 1.295$$

Resultat : $z_{\text{out}} = 1.295$

Etape 4 : Application de $y = \text{Sigmoid}(z_{\text{out}})$

La fonction Sigmoid est definie par Sigmoid

$$(z) = 1 / (1 + \exp(-z)).$$

En utilisant l'approximation $\exp(-1.295)$ environ egal a 0.274 :

$$y = 1 / (1 + 0.274) = 1 / 1.274 \text{ environ egal a } 0.785$$

Sortie finale du reseau : y environ egal a 0.785

Question 1.2 - Nombre de parametres

Formule utilisee : Parametres = (entrees * neurones) + neurones (biais)

Couche cachee Dense(2, ReLU) :

Nombre d'entrees = 3, nombre de neurones = 2

$$\text{Parametres} = (3 * 2) + 2 = 8$$

Couche de sortie Dense(1, Sigmoid) :

Nombre d'entrees = 2, nombre de neurones = 1

$$\text{Parametres} = (2 * 1) + 1 = 3$$

Total = 8 + 3 = 11 parametres entrainables

Question 1.3 - Interpretation

a) Classe predite : La sortie y environ egal a 0.785 est superieure au seuil de decision de 0.5. La classe predite est donc la classe 1 (positive).

b) Mecanisme d'ajustement : Le mecanisme est la Retropropagation du gradient (Backpropagation). Son principe consiste a calculer l'erreur (Loss) entre la valeur predite et reelle, puis a utiliser la regle de la chaine (Chain Rule) pour modifier les poids afin de reduire cette erreur.

Exercice 2 - Construction d'un modele Keras : prevision de la consommation electrique

Question 2.1 - Code Keras complet

```
import numpy as np
from sklearn.preprocessing import StandardScaler
from tensorflow import keras

# ===== PREPARATION DES DONNEES =====
# Donnees historiques (Temperature, Foyers, Mois, Tarif)
X_train = np.array([
    [8, 1200, 1, 4.5], [10, 1200, 2, 4.5], [14, 1250, 3, 4.5],
    [18, 1250, 4, 4.5], [24, 1300, 5, 4.5], [32, 1300, 6, 5.2],
    [38, 1350, 7, 5.2], [36, 1350, 8, 5.2], [28, 1300, 9, 4.5],
    [20, 1300, 10, 4.5], [12, 1250, 11, 4.5], [7, 1250, 12, 4.5]
])

# Consommation (MWh)
y_train = np.array([580, 520, 430, 350, 310, 480, 620, 590, 380, 330, 450, 600])

# ===== NORMALISATION =====
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)

# ===== CONSTRUCTION DU MODELE =====
model = keras.Sequential([
    keras.layers.Dense(16, activation='relu', input_shape=(4,)),
    keras.layers.Dense(8, activation='relu'),
    keras.layers.Dense(1, activation='linear')
])

# ===== COMPILATION =====
model.compile(optimizer='adam', loss='mean_squared_error', metrics=['mae'])

# ===== ENTRAINEMENT =====
model.fit(X_train_scaled, y_train, epochs=100, verbose=0)
```

```
# ===== PREDICTION (Scenarios A et B) =====
X_new = np.array([
    [35, 1400, 7, 5.2], # Scenario A
    [9, 1300, 12, 4.5] # Scenario B
])
X_new_scaled = scaler.transform(X_new)
predictions = model.predict(X_new_scaled)

print('Scenario A - Consommation prevue:', predictions[0][0])
print('Scenario B - Consommation prevue:', predictions[1][0])
```

Question 2.2 - Normalisation et parametres

a) Pourquoi la normalisation est indispensable :

Les variables ont des échelles très différentes : la température (7 à 38), le nombre de foyers (1200 à 1400) et le tarif (4.5 à 5.2).

Sans normalisation, la variable "Foyers" dominerait le calcul à cause de ses valeurs élevées, ce qui empêcherait l'optimiseur de converger correctement.

b) Calcul des parametres :

Couche 1 - Dense(16) : $(4 \text{ entrees} * 16 \text{ neurones}) + 16 \text{ biais} = 64 + 16 = 80 \text{ parametres}$

Couche 2 - Dense(8) : $(16 \text{ entrees} * 8 \text{ neurones}) + 8 \text{ biais} = 128 + 8 = 136 \text{ parametres}$

Couche 3 - Dense(1) : $(8 \text{ entrees} * 1 \text{ neurone}) + 1 \text{ biais} = 8 + 1 = 9 \text{ parametres}$

Total = 80 + 136 + 9 = 225 parametres entrainables.

Question 2.3 - Interpretation et avis argumente

a) Coherence des predictions :

Scenario A : La prédiction semble cohérente car une température élevée (35°C) et un grand nombre de foyers (1400) en juillet correspondent aux pics de consommation estivaux (utilisation intensive de la climatisation).

Scenario B : La prédiction est également cohérente car en décembre, malgré un tarif plus bas, la consommation augmente naturellement pour le chauffage et l'éclairage.

b) Avis argumente:

Ce modèle n'est pas suffisant pour un déploiement réel chez Sonelgaz.

Premièrement, 12 observations sont insuffisantes pour entraîner un modèle de 225 paramètres, ce qui mène inévitablement à un surapprentissage (overfitting).

Deuxièmement, des variables cruciales manquent comme l'activité industrielle, les jours fériés ou le taux d'humidité.

Troisièmement, une structure de type RNN ou LSTM serait plus adaptée pour traiter ces données temporelles.

Améliorations proposées : (1) Augmenter la base de données à plusieurs années de mesures horaires et (2) Intégrer des techniques de régularisation (Dropout) pour stabiliser les prédictions.

Exercice 3 - Diagnostic d'entrainement et regularisation

Question 3.1 - Diagnostic

Modele X (Train Loss = 0.42 / Val Loss = 0.45) :

Diagnostic : Sous-apprentissage (Underfitting).

Justification : Les deux pertes sont élevées avec un écart très faible, ce qui signifie que le modèle n'a pas réussi à capturer les tendances fondamentales des données.

Solutions : (1) Augmenter la complexité du modèle (plus de neurones ou de couches) et (2) Augmenter le nombre d'époques d'entraînement.

Modele Y (Train Loss = 0.02 / Val Loss = 0.85) :

Diagnostic : Surapprentissage (Overfitting).

Justification : La perte d'entraînement est très faible alors que la perte de validation est très élevée. Le modèle a mémorisé les données au lieu de généraliser.

Solutions : (1) Ajouter des couches de Dropout et (2) Utiliser le Early Stopping.

Modele Z (Train Loss = 0.05 / Val Loss = 0.09) :

Diagnostic : Modele satisfaisant.

Justification : Les deux pertes sont faibles et proches l'une de l'autre, indiquant un bon apprentissage et une bonne capacité de généralisation.

Question 3.2 - Correction par le code

```
from tensorflow import keras
from tensorflow.keras.callbacks import EarlyStopping
```

```
# ===== CONSTRUCTION DU MODELE CORRIGE =====
```

```
model = keras.Sequential([
    keras.layers.Dense(128, activation='relu', input_shape=(15,)),
    keras.layers.Dropout(0.4), # Ajout Dropout
    keras.layers.Dense(64, activation='relu'),
    keras.layers.Dropout(0.4), # Ajout Dropout
    keras.layers.Dense(1, activation='sigmoid')
])
```

```
# ===== COMPILATION =====
```

```
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
```

```
# ===== EARLY STOPPING =====
```

```
early_stop = EarlyStopping(  
    monitor='val_loss',  
    patience=10,  
    restore_best_weights=True  
)
```

```
# ===== ENTRAÎNEMENT AVEC VALIDATION =====
```

```
model.fit(  
    X_train, y_train,  
    epochs=200,  
    batch_size=16,  
    validation_split=0.2,  
    callbacks=[early_stop]  
)
```

Question 3.3 - Comprehension

a) Pourquoi le Dropout est désactivé lors de la prédiction :

Le Dropout est désactivé lors de la prédiction car celle-ci nécessite l'utilisation de tous les neurones pour obtenir un résultat stable et reproductible. S'il restait actif, les prédictions seraient aléatoires et incohérentes.

b) Role de patience=10 et restore_best_weights=True :

Patience=10 : Permet de continuer l'entraînement pendant 10 époques après la dernière amélioration pour surmonter des fluctuations temporaires.

Restore_best_weights=True : Restaure automatiquement les poids de l'époque où la val_loss était la plus basse.

Si False : Le modèle conserverait les poids de la dernière époque, qui sont souvent moins bons à cause du début du surapprentissage